

ACTL3142 Tutorial

Week 9: Tree-Based Methods

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T2 2026

Recap: Everything So Far Has Been Linear in β

- ▶ **Week 3 (MLR):** $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon$. Linear in the predictors *and* the coefficients.
- ▶ **Week 7 (Ridge/Lasso):** the same model, plus a penalty on the size of β , tuned by CV.
- ▶ **Week 8 (Splines, GAMs):** $y = \beta_0 + \sum_k \beta_k b_k(x) + \varepsilon$. The curve bends, but we only swapped x for **fixed** transformations of x : still one global equation fitted by least squares, still linear in β .

A tree has **no β , and no global equation at all.**

Trees are a basis expansion too, just a learned one

A tree is $\hat{f}(\mathbf{x}) = \sum_{m=1}^J c_m I(\mathbf{x} \in R_m)$: the same shape as Week 8, except the regions R_m are **chosen by looking at the data**. That freedom yields interactions automatically, at the cost of making a single tree unstable.

Decision Trees: The Idea

A decision tree splits the predictor space into **non-overlapping rectangular regions** $R_1, \dots, R_J$ using a sequence of binary rules.

Regression: predict $\hat{y} = \bar{y}_{R_j}$, the mean of the training observations in that region.

Classification: predict the **most common class** in the region.

Why trees are appealing:

- ▶ Easy to interpret and visualise (mirrors human decision-making)
- ▶ Handle interactions and non-linearities naturally
- ▶ Take qualitative predictors directly, with no assumption on the form of $f(X)$

Downsides

A single tree has **high variance**: move a few observations and you can get a completely different tree. This motivates pruning and, later, ensemble methods.

Growing a Regression Tree: Recursive Binary Splitting

We need a **loss function** to decide where to split. For regression trees we use **squared error**, with each region predicting $\hat{y}_{R_j} = \bar{y}_{R_j}$.

Finding the globally optimal partition is infeasible, so we go **greedy and top-down**:

1. Consider all predictors X_j and all cutpoints s
2. Pick the (j, s) minimising the combined RSS of the two child nodes:

$$\sum_{i: \mathbf{x}_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: \mathbf{x}_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

where $R_1(j, s) = \{X \mid X_j < s\}$ and $R_2(j, s) = \{X \mid X_j \geq s\}$

3. Repeat on **one of the two new regions**, until a stopping rule is hit (e.g. every leaf has fewer than 5 observations)

“Greedy” means each split is locally optimal: we never check whether a worse split now would allow a better tree later. Same idea as forward stepwise selection.

Classification Trees: Loss Functions

RSS no longer applies, so we need a loss measuring **node impurity**. Let $\hat{p}_{mk}$ be the proportion of class k in node m :

Loss function	Formula	Use it for
Classification error	$1 - \max_k(\hat{p}_{mk})$	Pruning, reporting
Gini index	$\sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$	Growing
Entropy	$-\sum_{k=1}^K \hat{p}_{mk} \ln(\hat{p}_{mk})$	Growing

All three are zero at a pure node and maximised when the classes are evenly mixed. But each is a **per-observation** loss, so a split is scored by weighting each child by how many observations land in it:

$$\Delta = Q(\text{parent}) - \frac{n_L}{n} Q(L) - \frac{n_R}{n} Q(R),$$

and we take the (j, s) with the largest Δ . Regression hides this step: RSS is already summed over observations, so the node sizes are baked in.

Why not grow the tree with classification error?

It only tracks the majority class, so a split can make both children purer without moving it at all. Gini and entropy respond to *any* change in the class proportions, so they can still rank those splits.

Cost-Complexity Pruning: The Criterion

A fully grown tree overfits. Rather than stopping early, **grow a large tree T_0 first, then prune back**: a weak split now might unlock a great split below it, and early stopping never finds out.

So add a **penalty for tree size**:

$$C_\alpha(T) = \underbrace{\sum_{m=1}^{|T|} \sum_{i: \mathbf{x}_i \in R_m} (y_i - \hat{y}_{R_m})^2}_{\text{fit, } = R(T)} + \underbrace{\alpha |T|}_{\text{complexity}}$$

$|T|$ is the number of leaves and $\alpha \geq 0$ is the tuning parameter: $\alpha = 0$ keeps T_0 , large α collapses the tree toward its root.

Interpreting α

α acts as a price per leaf: a leaf is retained only if it reduces RSS by more than α . Raising α removes the least productive leaves first.

Branches are pruned in a **nested** sequence $T_0 \supset T_1 \supset \dots \supset \{\text{root}\}$, each tree optimal over an interval of α , so we choose among only $|T_0|$ candidates by cross-validation.

Exactly the shape of ridge and lasso, with complexity counted in leaves.

Choosing α by Cross-Validation

Each row of `rpart`'s `cptable` is one tree in that nested sequence.
From the lecture's Hitters example:

CP	nsplit	$ T $	rel error	xerror	xstd
0.4446	0	1	1.0000	1.0072	0.0657
0.1145	1	2	0.5554	0.5640	0.0590
0.0445	2	3	0.4409	0.4620	0.0575
0.0183	3	4	0.3964	0.4290	0.0594
0.0169	4	5	0.3781	0.4330	0.0639

rel error is training error; xerror is **cross-validated** error, which does not fall forever.

- ▶ **Minimum** xerror is 0.4290 at 3 splits, so **4 leaves**.
- ▶ **1-SE rule**: smallest tree within one xstd of the minimum. Here $0.4290 + 0.0594 = 0.4884$, which 2 splits clears, so **3 leaves**, matching the lecture.

rpart's cp is $\alpha/R(\text{root})$, on a 0–1 scale (default 0.01, where this table stops).

Why Ensemble Methods?

A single decision tree is:

- + Interpretable and intuitive
- High variance: small changes in the data give a very different tree
- Often poor predictive accuracy compared to other methods

Key insight: averaging many high-variance, low-bias models **reduces variance** while keeping bias low.

If $Z_1, \dots, Z_B$ are i.i.d. with variance σ^2 , then $\text{Var}(\bar{Z}) = \sigma^2/B$.

Three ensemble approaches:

- ▶ **Bagging:** many deep trees on bootstrap samples, averaged (variance reduction)
- ▶ **Random forests:** bagging + randomise the predictors available at each split, to decorrelate the trees
- ▶ **Boosting:** small trees grown sequentially, each correcting what the ensemble still gets wrong (bias reduction)

Bagging (Bootstrap Aggregation)

Idea: we cannot collect many independent training sets, but we *can* simulate them by **bootstrapping**.

Procedure:

1. Draw B bootstrap samples (n observations, **with replacement**)
2. Fit a **deep, unpruned** tree to each one
3. Aggregate: regression $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(\mathbf{x})$; classification, majority vote

Why not prune the individual trees?

Deep trees are low bias, high variance. Averaging is what removes the variance, so each tree is left as flexible as possible and the averaging handles the variance.

Increasing B never causes overfitting: more trees just reduce variance further, with diminishing returns. B is not a tuning parameter, just pick it large enough.

Out-of-Bag (OOB) Error

Bootstrapping hands us a validation set for free. The probability that observation i misses a given bootstrap sample is

$$\left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368,$$

so each tree trains on about two-thirds of the data and leaves the rest **out of bag**.

The procedure, precisely:

1. For observation i , collect the trees for which i was OOB (about $B/3$ of them)
2. Average their predictions, or take their majority vote, to get $\hat{y}_i^{\text{OOB}}$
3. OOB error = $\frac{1}{n} \sum_i (y_i - \hat{y}_i^{\text{OOB}})^2$, or the misclassification rate

It is honest because no tree used in step 2 ever saw y_i , and it costs **one** fit where k -fold CV costs k .

Slightly pessimistic: each $\hat{y}_i^{\text{OOB}}$ uses only $\sim B/3$ trees, and a smaller ensemble is a worse one. “With B sufficiently large, OOB error is virtually equivalent to LOOCV” (James et al., 2021).

Random Forests

Problem with bagging: if one predictor is very strong, *every* bootstrapped tree splits on it first. The trees end up highly **correlated**, and averaging correlated quantities does not reduce variance effectively.

The fix: at each split, only consider a **fresh random subset** of m predictors out of p .

- ▶ Typical choice: $m \approx \sqrt{p}$ (classification), $m \approx p/3$ (regression)
- ▶ The strong predictor is unavailable at most splits, so weaker predictors are used more often
- ▶ When $m = p$, a random forest **is** bagging

Why decorrelation helps

With pairwise correlation ρ between trees, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Only the second term shrinks with B , so once B is large the first term is **all that is left**. Reducing ρ is the only way to attack it.

Variable Importance

We lose the single-tree picture: hundreds of trees, each using different predictors and leaves. But we can still rank the predictors, in two quite different ways.

1. Loss reduction (`IncNodePurity`): for each predictor, add up the reduction in RSS (regression) or Gini index (classification) over every split using it, averaged over the B trees. Free with the fit, but biased toward continuous and many-levelled predictors, which get offered more candidate splits.

2. Permutation importance (`%IncMSE`): shuffle predictor j in the OOB data and see how much the OOB error rises. If the model relied on j , breaking it hurts. Measured on held-out data, so negative values (as in the lecture's flood data) just mean no signal.

In R: `importance(rf)` gives both columns (needs `importance = TRUE` when fitting) and `summary(gbm_fit)` gives boosting's "relative influence".

Careful with correlated predictors: they share the credit, so a useful variable can look unimportant. This ranks predictive contribution, not causal effect.

Boosting: The Idea

Bagging and random forests build B trees **in parallel** on resampled data, then average them. Boosting builds **one** model, **incrementally**:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^B \lambda \hat{f}^b(\mathbf{x}),$$

where each $\hat{f}^b$ is a small correction. After b trees, look at what the ensemble still gets wrong (the **residuals** r_i) and aim the next tree at those. No bootstrapping: each tree sees all the data, with a new response each round.

Why deliberately weak learners?

Fit a *deep* tree to the residuals in round 1 and it drives them to nearly zero at once, leaving one overfit tree and nothing for rounds 2 to B to do. Boosting works only if every step is constrained: few splits (d small) and a small step size (λ small). That is “slow learning”.

Boosting: The Algorithm (Regression Trees)

1. Set $\hat{f}(\mathbf{x}) = 0$ and $r_i = y_i$ for all i .
2. For $b = 1, 2, \dots, B$:
 - (a) Fit a tree $\hat{f}^b$ with d splits (so $d + 1$ leaves) to the data $(X, \mathbf{r})$
 - (b) Add a **shrunk** version of it: $\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}^b(\mathbf{x})$
 - (c) Update the residuals: $r_i \leftarrow r_i - \lambda \hat{f}^b(\mathbf{x}_i)$
3. Output $\hat{f}(\mathbf{x}) = \sum_{b=1}^B \lambda \hat{f}^b(\mathbf{x})$.

The response in step (a) is $\mathbf{r}$, not $\mathbf{y}$. After round 1 the trees never see the original response again.

What one round with a stump ($d = 1$) does: find the single split, over all predictors and cutpoints, that best explains the *current* residuals; predict the mean residual in each of its two leaves; then move a fraction λ of the way there.

Same greedy split search as an ordinary tree. The only change is what it is fitted to.

Boosting: A Quick Example

Four observations, stumps ($d = 1$), learning rate $\lambda = 0.5$.

	$x = 1$	$x = 2$	$x = 3$	$x = 4$
y	0	0	12	24
Round 1: residuals $\mathbf{r}$	0	0	12	24
best stump is $x < 3$, giving $\hat{f}^1$	0	0	18	18
$\hat{f} \leftarrow \hat{f} + 0.5 \hat{f}^1$	0	0	9	9
Round 2: residuals $\mathbf{r}$	0	0	3	15
best stump is $x < 4$, giving $\hat{f}^2$	1	1	1	15
$\hat{f} \leftarrow \hat{f} + 0.5 \hat{f}^2$	0.5	0.5	9.5	16.5

Round 1 has one cut, so it must lump $x = 3$ and $x = 4$ together at 18. Round 2 reads the *residuals* and finds the split round 1 could not: two stumps give a **three-region** step function.

Is fitting residuals really fitting a gradient?

Yes. With squared-error loss $L = \frac{1}{2}(y - \hat{f})^2$, $\partial L / \partial \hat{f} = -(y - \hat{f})$, so the residual **is** the negative gradient and step (b) is a gradient-descent step of size λ . Other losses replace $\mathbf{r}$ with their own negative gradient, hence **gradient boosting**.

Boosting: Two More Rounds

Each round is the **same** procedure: grow a one-split tree using the current $\mathbf{r}$ as the response, take the mean residual in each leaf, add λ times it.

	$x = 1$	$x = 2$	$x = 3$	$x = 4$
Round 3: residuals $\mathbf{r}$	-0.5	-0.5	2.5	7.5
best stump is $x < 4$, giving $\hat{f}^3$	0.5	0.5	0.5	7.5
$\hat{f} \leftarrow \hat{f} + 0.5 \hat{f}^3$	0.75	0.75	9.75	20.25
Round 4: residuals $\mathbf{r}$	-0.75	-0.75	2.25	3.75
best stump is $x < 3$, giving $\hat{f}^4$	-0.75	-0.75	3	3
$\hat{f} \leftarrow \hat{f} + 0.5 \hat{f}^4$	0.375	0.375	11.25	21.75
Target y , for comparison	0	0	12	24

Training RSS falls monotonically: $720 \rightarrow 234 \rightarrow 63 \rightarrow 20.25 \rightarrow 5.91$.

Round 3 made $x = 1, 2$ worse

One cut forces $\{-0.5, -0.5, 2.5\}$ into a single leaf averaging $+0.5$, pushing those two points *further* from zero. Total RSS still fell, because the gain at $x = 4$ outweighed it, and round 4 corrected them. A weak learner is not required to improve every observation, only the loss overall.

Boosting: The Three Tuning Parameters

B , the number of trees. Boosting can overfit if B is too large.

Choose it by CV (`gbm.perf(..., method = "cv")`) or early stopping on a validation set.

*Why B overfits here but not in bagging: in bagging every tree independently estimates the same target, so the average converges. In boosting every tree **changes the model**, so complexity keeps growing with B .*

λ , the shrinkage or learning rate. Typically 0.01 or 0.001. It trades off against B : halve λ and you roughly need to double B .

d , the splits per tree (giving $d + 1$ leaves). $d = 1$ is the standard starting point; the lecture's flood insurance demo used $d = 2$.

In `gbm` this argument is called `interaction.depth`, for the reason on the next slide.

Choosing them in practice

B and λ are chosen together: fix a small λ , then use CV to find how many trees it needs.

*In practice **XGBoost** and **LightGBM** are standard, not `gbm`: each tree is fitted to the **negative** gradient of a chosen loss, with extra regularisation. Not covered in this course.*

Interaction Depth: Why $d = 1$ Gives a GAM

A stump splits on **one** variable, so each $\hat{f}^b$ is a function of that variable alone. Group the B stumps by the variable they split on:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^B \lambda \hat{f}^b(\mathbf{x}) = \sum_{j=1}^p g_j(x_j), \quad g_j(x_j) = \lambda \sum_{b: \text{splits on } x_j} \hat{f}^b(\mathbf{x}).$$

That is exactly the **GAM** from Week 8, $\hat{f} = \beta_0 + \sum_j g_j(x_j)$, except each g_j is a step function built from shrunken stumps rather than a spline.

Why this caps interactions

No term in $\sum_j g_j(x_j)$ involves two variables at once, so with $d = 1$ the effect of x_1 cannot depend on x_2 , however large B gets. With $d = 2$ a single tree can split on x_1 and then on x_2 below it, producing a region $\{x_1 < s_1\} \cap \{x_2 < s_2\}$: a genuine two-way interaction.

So d sets the maximum order of interaction the model can express. Use $d = 1$ for an additive, interpretable fit; raise it when you expect predictors to act jointly.

Bagging vs Random Forests vs Boosting

	Bagging	Random Forest	Boosting
Trees built	Independently	Independently	Sequentially
Fitted to	Bootstrap data	Bootstrap data	Current residuals
Individual trees	Deep (low bias)	Deep (low bias)	Small (stumps)
Improves by	Less variance	Less variance	Less bias
Predictors/split	All p	Random $m < p$	All p
B can overfit?	No	No	Yes
Free OOS error	OOB	OOB	No, use CV

All three trade the single-tree picture for accuracy, and all three give variable importance back in exchange.

Boosting often gives the best predictive performance, but it is the only one of the three that genuinely needs careful tuning. Bagging and random forests work well close to out of the box.

Q1: Gini Index, Classification Error, and Entropy

★ [Solution](#) (ISLR2, Q8.3) Consider the Gini index, classification error, and entropy in a simple classification setting with two classes. Create a single plot that displays each of these quantities as a function of $\hat{p}_{m1}$. The x -axis should display $\hat{p}_{m1}$, ranging from 0 to 1, and the y -axis should display the value of the Gini index, classification error, and entropy.

1. **Hint:** In a setting with two classes, $\hat{p}_{m1} = 1 - \hat{p}_{m2}$. You could make this plot by hand, but it will be much easier to make in [R](#).

Q1: Walkthrough

For two classes ($K = 2$), let $p = \hat{p}_{m1}$, so $\hat{p}_{m2} = 1 - p$:

- ▶ **Classification error:** $E = 1 - \max(p, 1 - p)$, piecewise linear (an inverted V), peaking at $p = 0.5$ with value 0.5
- ▶ **Gini index:** $G = 2p(1 - p)$, a downward parabola, peaking at $p = 0.5$ with value 0.5
- ▶ **Entropy:** $D = -p \ln(p) - (1 - p) \ln(1 - p)$, peaking at $p = 0.5$ with value $\ln 2 \approx 0.693$

All three are zero at $p = 0$ and $p = 1$ (a pure node) and maximised at $p = 0.5$ (maximum impurity).

The **shape** is the point. Gini and entropy are strictly concave, so moving a node from $p = 0.9$ to $p = 0.95$ registers as a real improvement. Classification error has constant slope on each side, so near the extremes it often cannot separate a good split from a bad one.

In R: `plot all three over  $p \leftarrow \text{seq}(0, 1, \text{length} = 100)$ .`

Q2: Tree and Partition Correspondence

★ [Solution](#) (ISLR2, Q8.4) This question relates to the plots in the textbook Figure 8.14, reproduced here as [Figure 1](#):

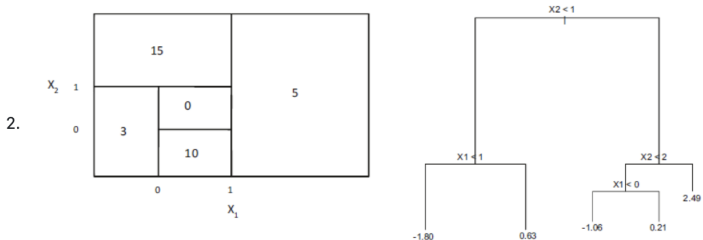
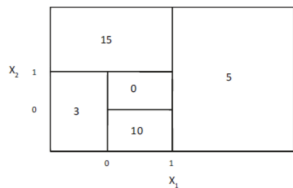


Figure 1: Left: A partition of the predictor space corresponding to Exercise 4a. Right: A tree corresponding to Exercise 4b.

- Sketch the tree corresponding to the partition of the predictor space illustrated in the left-hand panel of Figure 8.14. The numbers inside the boxes indicate the mean of Y within each region.
- Create a diagram similar to the left-hand panel of Figure 8.14, using the tree illustrated in the right-hand panel of the same figure. You should divide up the predictor space into the correct regions, and indicate the mean for each region.

Q2: Walkthrough

(a) **Partition** $\rightarrow$ **tree**. Each boundary line is one split. Start from the line cutting the **whole** space and work down.



$X_1 < 1$?

No $\rightarrow \hat{y} = 5$

Yes: $X_2 < 1$?

No $\rightarrow \hat{y} = 15$

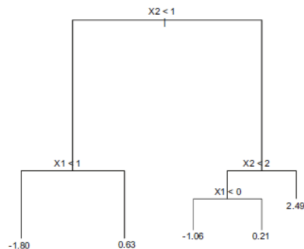
Yes: $X_1 < 0$?

Yes $\rightarrow \hat{y} = 3$

No: $X_2 < 0$?

Yes $\rightarrow \hat{y} = 10$, **No** $\rightarrow \hat{y} = 0$

(b) **Tree** $\rightarrow$ **partition**. Trace each root-to-leaf path; each one defines a rectangle.



- ▶ $X_2 < 1, X_1 < 1 \rightarrow \hat{y} = -1.80$
- ▶ $X_2 < 1, X_1 \geq 1 \rightarrow \hat{y} = 0.63$
- ▶ $1 \leq X_2 < 2, X_1 < 0 \rightarrow \hat{y} = -1.06$
- ▶ $1 \leq X_2 < 2, X_1 \geq 0 \rightarrow \hat{y} = 0.21$
- ▶ $X_2 \geq 2 \rightarrow \hat{y} = 2.49$

In (a), the vertical line at $X_1 = 1$ runs the full height, so it must be the root: only the root split can cut the entire space. In (b), every leaf is exactly one rectangle, and the top region $X_2 \geq 2$ is never split on X_1 , so it spans the full width.

Q3: Majority Vote vs Average Probability

3. ★ [Solution](#) (ISLR2, Q8.5) Suppose we produce ten bootstrapped samples from a data set containing red and green classes. We then apply a classification tree to each bootstrapped sample and, for a specific value of X , produce 10 estimates of $\mathbb{P}(\text{Class is Red}|X)$:

0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, and 0.75.

There are two common ways to combine these results together into a single class prediction. One is the majority vote approach discussed in this chapter. The second approach is to classify based on the average probability. In this example, what is the final classification under each of these two approaches?

Q3: Walkthrough

Ten bootstrapped trees give these estimates of $P(\text{Red} \mid X)$:
0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75

Majority vote: each tree votes Red if $P > 0.5$, Green otherwise.

- ▶ Red: 6 trees (0.55, 0.6, 0.6, 0.65, 0.7, 0.75)
- ▶ Green: 4 trees (0.1, 0.15, 0.2, 0.2)
- ▶ Final classification: **Red** (6 vs 4)

Average probability:

$$\bar{P} = \frac{0.1 + 0.15 + \dots + 0.75}{10} = \frac{4.5}{10} = 0.45 < 0.5 \Rightarrow \text{Green.}$$

Why do they disagree?

Majority vote throws away *how confident* each tree is, turning every probability into a 0 or a 1 before averaging. The four Green trees are very confident (0.1 to 0.2), while most Red trees sit barely above 0.5. Averaging the probabilities keeps that information.

Applied Q1: Trees and Ensembles on Carseats

Pull this question up on your side. Key pointers for each part:

- (a) `set.seed()`, then split into train/test (the solution uses 50/50).
- (b) `tree(Sales ~ ., data = Carseats, subset = train)`, then `plot()` and `text(fit, pretty = 0)`. Test MSE ≈ 4.92 . The base plot is hard to read: `rpart()` with `rpart.plot()` is far more readable. Top split is `ShelveLoc`, then `Price`.
- (c) `cv.tree(fit, FUN = prune.tree)`. Check `$size`, `$dev` and `$k`: that `$k` is the α from the pruning slides. Minimum deviance at 5 leaves, $\alpha \approx 106.9$. Then `prune.tree(fit, best = 5)`.
- (d) Bagging: `randomForest(..., mtry = ncol(Carseats) - 1, importance = TRUE)`. Test MSE ≈ 2.65 ; `importance()` puts `Price` and `ShelveLoc` on top.
- (e) Random forest: default `mtry = p/3 \approx 3`, test MSE ≈ 3.03 . Loop `mtry` from 1 to 10 and plot test MSE.
- (f) Optional BART. Not covered in this course, so skip it.

Note bagging beats the random forest here: with $p = 10$ and two dominant predictors, $m = 3$ excludes them too often for the decorrelation gain to cover the loss. Also, the posted solution runs `cv.tree()` on the full data rather than the training set.

Applied Q3: Boosting on Hitters

Pull this question up on your side. Key pointers for each part:

- (a) Drop rows with missing Salary, then log-transform it.
- (b) Training set = first 200 observations, test set = the rest.
- (c) `gbm(..., distribution = "gaussian", n.trees = 1000, shrinkage = lambda[i])` over a grid, e.g. `10^seq(-5, -0.2, by = 0.05)`. Plot training MSE against λ : it falls monotonically, since bigger steps fit the training data harder.
- (d) Same loop, now test MSE against λ . This one is **U-shaped**: too small and 1000 trees is not enough to learn, too large and it overfits. Read your own minimum off the plot.
- (e) Compare against two Chapter 3/6 methods: stepwise `lm` (test MSE ≈ 0.493) and lasso via `cv.glmnet` (≈ 0.476). Boosting beats both at a sensible λ .
- (f) `summary(fit)` gives relative influence: `CAtBat` and `PutOuts` come out on top.
- (g) Bagging: `randomForest(..., mtry = ncol - 1)`. Test MSE ≈ 0.230 .

Bagging wins here, which is worth noting. We only tuned λ : B was fixed at 1000 and d left at its default. Boosting's performance depends heavily on tuning, exactly the tradeoff in the comparison table.

Thanks for coming! See you next week.

